T02

PROGRAMACIÓN ORIENTADA A OBJETOS:CLASES Y OBJETOS (I)

INTRODUCCIÓN

La POO es un paradigma surgido en los años 1970, que utiliza objetos como elementos fundamentales en la construcción de la solución. Un objeto es una abstracción de algún hecho o ente del mundo real, con atributos que representan sus características o propiedades, y métodos que emulan su comportamiento o actividad. Todas las propiedades y métodos comunes a los objetos se encapsulan o agrupan en clases. Una clase es una plantilla, un prototipo para crear objetos; en general, se dice que cada objeto es una instancia o ejemplar de una clase.

¿QUÉ ES LA PROGRAMACIÓN ORIENTADA A OBJETOS?

- La programación orientada a objetos (OOP) es un paradigma de programación que parte del concepto de "objetos" como base, los cuales contienen información en forma de atributos y código en forma de métodos.
    - variable → atributos
    - función → método
- Los objetos son capaces de interactuar y modificar los valores contenidos en sus atributos (estado) a través de sus métodos (comportamiento).
- Lenguajes Orientados a Objetos:
    - Simula (1967)
    - Smalltalk (1972 a 1980)
    - Python
    - Java
    - C++
    - JavaScript
    - C#
    - Ruby
    - TypeScript
    - Swift
    - ...

CARACTERÍSTICAS DE LA OOP

- **Abstracción**: Cada objeto en el sistema sirve como modelo de un "agente" abstracto que puede realizar trabajo, informar y cambiar su estado, y "comunicarse" con otros objetos en el sistema sin revelar "cómo" se implementan estas características.

- **Encapsulamiento**: Significa reunir todos los elementos que pueden considerarse pertenecientes a una misma entidad, al mismo nivel de abstracción, protegiéndolos del acceso directo desde el exterior.

- **Polimorfismo**: Comportamientos diferentes, asociados a objetos distintos, pueden compartir el mismo nombre; al llamarlos por ese nombre se utilizará el comportamiento correspondiente al objeto que se esté usando.

- **Herencia**: Las clases no se encuentran aisladas, sino que se relacionan entre sí, formando una jerarquía. Los objetos heredan las propiedades y el comportamiento de todas las clases a las que pertenecen.

PROGRAMACIÓN ORIENTADA A OBJETOS EN PYTHON

DEFINIENDO CLASES EN PYTHON

```
class NombreDeLaClase:
    # Cuerpo de la clase
    pass
```

DEFINIENDO CLASES EN PYTHON

- En el cuerpo de una clase, puede definir atributos y métodos según sea necesario. Los atributos son variables que contienen los datos de la clase, mientras que los métodos son funciones que proporcionan comportamiento y normalmente actúan sobre los datos de la clase.

- En Python, el cuerpo de una clase determinada funciona como un espacio de nombres (namespace) donde residen los atributos y métodos. Sólo podemos acceder a esos atributos y métodos a través de la clase o sus objetos.

- Como ejemplo de cómo definir atributos y métodos, crearemos la clase Circulo para modelar diferentes círculos en una aplicación de dibujo.

- Inicialmente, la clase tendrá un único atributo para mantener el radio. También tendrá un método para calcular el área del círculo.

DEFINIENDO CLASES EN PYTHON

```
# circulo.py

import math

class Circulo:
    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return round(math.pi * \
            self.radio ** 2, 2)
```

Atributo

Constructor

Método

En Python, el primer argumento de la mayoría de los métodos es *self*. Este argumento contiene una referencia al objeto actual para que podamos usarlo dentro de la clase.

CREANDO OBJETOS DE UNA CLASE

```
>>> from circulo import Circulo

>>> circulo_1 = Circulo(42)
>>> circulo_2 = Circulo(7)

>>> circulo_1
<__main__.Circulo object at
0x102b835d0>

>>> circulo_2
<__main__.Circulo object at
0x1035e3910>
```

self?

La acción de crear objetos concretos a partir de una clase existente se conoce como instanciación.

Al crear un objeto, se ejecuta automáticamente el constructor __init__(self, ...)

ACCEDIENDO A ATRIBUTOS Y MÉTODOS

```
>>> from circulo import Circulo

>>> circulo_1 = Circulo(42)
>>> circulo_2 = Circulo(7)

>>> circulo_1.radio
42
>>> circulo_1.calcular_area()
5541.77

>>> circulo_2.radio
7
>>> circulo_2.calcular_area()
153.94

>>> circulo_1.radio = 100
>>> circulo_1.radio
100
>>> circulo_1.calcular_area()
31415.93
```

EJERCICIO 2.1

CONVENCIONES DE NOMENCLATURA EN CLASES DE PYTHON

- Python es un lenguaje flexible y no tiene restricciones explícitas en cuanto al nombre de las variables. Por eso, el lenguaje y la comunidad se basan en convenciones en lugar de restricciones.

- La mayoría de los programadores de Python siguen la convención de nomenclatura **snake_case**, que implica el uso de guiones bajos (_) para separar varias palabras.

- Sin embargo, la convención de nomenclatura recomendada para las clases de Python es **PascalCase**, donde cada palabra está en mayúscula.

CUÁNDO NO USAR CLASES

- Almacenar solo datos. Utilizaremos el decorador *@dataclass* (veremos decoradores en Tema 14) o una tupla en su lugar.

- Proporcionar un método único. Utilizaremos una función en su lugar.

- Un programa o script pequeño y simple que no requiere lógica o estructuras de datos complejas. En este caso, usar clases puede resultar excesivo.

- Un programa crítico para el rendimiento. Las clases agregan sobrecarga general a un programa, especialmente cuando necesitamos crear muchos objetos. Esto puede afectar el rendimiento general de su código.

- Código antiguo no orientado a objetos. Si un código existente no utiliza clases, entonces no deberíamos introducirlas. Esto romperá el estilo de programación actual y alterará la coherencia del código.

- Un equipo de trabajo con un estilo de codificación diferente. Si nuestro equipo de trabajo actual no usa clases, seguiremos con su estilo de codificación. Esto garantizará la coherencia en todo el proyecto.

AÑADIENDO DATOS A CLASES Y OBJETOS

- Primero, necesitamos saber que las clases pueden tener dos tipos de atributos en Python:

    - Atributos de clase: un atributo de clase es una variable que definimos directamente en el cuerpo de la clase. Los atributos de clase pertenecen a su clase contenedora. Sus datos son comunes a la clase y todas sus instancias.

    - Atributos de instancia: una instancia es una variable que se define dentro de un constructor. Los atributos de instancia pertenecen a una instancia concreta de una clase determinada. Sus datos solo están disponibles para esa instancia y definen su estado.

- Ambos tipos de atributos tienen sus casos de uso específicos. Los atributos de instancia son, con diferencia, el tipo de atributo más común que utilizaremos, pero los atributos de clase también resultan útiles.

ATRIBUTOS DE CLASE

- Los atributos de clase son variables que se definen directamente en el cuerpo de la clase, fuera de cualquier método. Estos atributos están vinculados a la clase misma y no a objetos particulares de esa clase.

- Todos los objetos que creamos a partir de una clase particular comparten los mismos atributos de clase con los mismos valores originales. Debido a esto, si cambiamos un atributo de clase, ese cambio afecta a todos los objetos derivados.

- Como ejemplo, vamos a crear una clase que mantenga un recuento interno de las instancias que ha creado. En ese caso, podemos usar un atributo de clase.

ATRIBUTOS DE CLASE

```
>>> class ContadorDeObjetos:
...     num_instancias = 0
...     def __init__(self):
...         ContadorDeObjetos.num_instancias += 1
...

>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x10392d810>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x1039810d0>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x10395b750>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x103959810>

>>> ContadorDeObjetos.num_instancias
4

>>> contador = ContadorDeObjetos()
>>> contador.num_instancias
5
```

ATRIBUTOS DE CLASE

```
>>> class ContadorDeObjetos:
...     num_instancias = 0
...     def __init__(self):
...         type(self).num_instancias += 1

>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at
0x10392d810>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at
0x1039810d0>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at
0x10395b750>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at
0x103959810>

>>> ContadorDeObjetos.num_instancias
4

>>> contador = ContadorDeObjetos()
>>> contador.num_instancias
5
```

evita hard-coding

ATRIBUTOS DE CLASE

```
>>> class ContadorDeObjetos:
...     num_instancias = 0
...     def __init__(self):
...         self.num_instancias += 1
...

>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x103987550>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x1039c5890>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x10396a890>
>>> ContadorDeObjetos()
<__main__.ContadorDeObjetos object at 0x1036fa110>

>>> ContadorDeObjetos.num_instancias
0
```

Si usamos self para modificar .num_instancias, sobrescribirá el atributo de clase original creando un nuevo atributo de instancia.

ATRIBUTOS DE INSTANCIA

- Los atributos de instancia son variables vinculadas a un objeto particular de una clase determinada. El valor de un atributo de instancia se adjunta al propio objeto. Así, el valor del atributo es específico de la instancia que lo contiene.

- Python permite adjuntar dinámicamente atributos a objetos existentes. Sin embargo, la mayoría de las veces debemos definir atributos de instancia dentro de métodos de instancia, que son aquellos métodos que reciben *self* como primer argumento.

- Aunque podemos definir atributos de instancia dentro de cualquier método de instancia, es mejor definirlos todos en el constructor *__init__*. Esto garantiza que todos los atributos tengan los valores correctos cuando cree una nueva instancia. Además, hace que el código esté más organizado y sea más fácil de depurar.

- Dentro de una clase, debemos acceder a todos los atributos de la instancia a través de *self*. Este argumento contiene una referencia a la instancia actual, que es donde están almacenados los atributos.

ATRIBUTOS DE INSTANCIA

```
# coche.py

class Coche:
    def __init__(self, marca, modelo, anyo, color):
        self.marca = marca
        self.modelo = modelo
        self.anyo = anyo
        self.color = color
        self.en_marcha = False
        self.velocidad = 0
        self.velocidad_maxima = 200
```

ATRIBUTOS DE INSTANCIA

```
>>> from coche import Coche

>>> toyota_camry = Coche("Toyota", "Camry", 2022, "Rojo")
>>> toyota_camry.marca
'Toyota'
>>> toyota_camry.modelo
'Camry'
>>> toyota_camry.color
Rojo'
>>> toyota_camry.velocidad
0

>>> ford_mustang = Coche("Ford", "Mustang", 2022, "Negro")
>>> ford_mustang.marca
'Ford'
>>> ford_mustang.modelo
'Mustang'
>>> ford_mustang.anyo
2022
>>> ford_mustang.velocidad_maxima
200

>>> Coche.marca
Traceback (most recent call last):
    ...
AttributeError: type object 'Coche' has no attribute 'marca'
```

EJERCICIO 2.2

ATRIBUTOS DINÁMICOS DE CLASE E INSTANCIA

- En Python, podemos agregar nuevos atributos a clases e instancias de forma dinámica. Esta posibilidad permite adjuntar nuevos datos y comportamientos a clases y objetos en respuesta a requisitos o contextos cambiantes. También permite adaptar las clases existentes a necesidades específicas y dinámicas.

- Por ejemplo, podemos aprovechar esta característica de Python cuando no conocemos los atributos requeridos de una clase determinada en el momento en que define esa clase.

- Podemos utilizar la notación de punto (Clase.miembro) y una asignación para agregar nuevos atributos y métodos a una clase de forma dinámica.

ATRIBUTOS DINÁMICOS DE CLASE E INSTANCIA

```
# mi_clase.py

class MiClase:
   def __init__(self, mi_atributo):
      self.mi_atributo = mi_atributo
```

```
>>> from mi_clase import MiClase

>>> MiClase.mi_atributo_dinamico_clase = 'Hola'

>>> MiClase.mi_atributo_dinamico_clase
'Hola'

>>> mi_objeto = MiClase('Mi Atributo Estatico')

>>> mi_objeto.mi_atributo_dinamico_instancia = 'Adios'

>>> mi_objeto.mi_atributo_dinamico_instancia
'Adios'
```

EL ATRIBUTO .__dict__

- En Python, tanto las clases como las instancias tienen un atributo especial llamado .__dict__
- Este atributo contiene un diccionario que contiene los miembros (atriburos y métodos) de la clase o instancia subyacente. Cada clave en .__dict__ representa un nombre de atributo. El valor asociado con una clave determinada representa el valor del atributo correspondiente.
- Cuando accedemos a un miembro de la clase a través del objeto de clase, Python busca automáticamente el nombre del miembro en el .__dict__ de la clase. Si el nombre no está allí, obtendremos un *AttributeError*.
- De manera similar, cuando accedemos a un miembro de una instancia a través de una instancia concreta de una clase, Python busca el nombre del miembro en el .__dict__ de la instancia. Si el nombre no aparece allí, entonces Python busca en el .__dict__ de la clase. Si no se encuentra el nombre, obtendremos un *NameError*.

EL ATRIBUTO .__dict__

```
# sample_dict.py

class SampleClass:
    class_attr = 100

    def __init__(self, instance_attr):
        self.instance_attr = instance_attr
```

EL ATRIBUTO .__dict__

```
>>> from sample_dict import SampleClass

>>> SampleClass.class_attr
100

>>> SampleClass.__dict__
mappingproxy({
    '__module__': '__main__',
    'class_attr': 100,
    '__init__': <function SampleClass.__init__ at 0x1036c62a0>,
    '__dict__': <attribute '__dict__' of 'SampleClass' objects>,
    '__weakref__': <attribute '__weakref__' of 'SampleClass' objects>,
    '__doc__': None
})

>>> SampleClass.__dict__["class_attr"]
100
```

EL ATRIBUTO .__dict__

```
>>> instance = SampleClass("Hello!")

>>> instance.instance_attr
'Hello!'

>>> instance.__dict__
{'instance_attr': 'Hello!'}

>>> instance.__dict__["instance_attr"]
'Hello!'

>>> instance.__dict__["instance_attr"] = "Hello, Python!"

>>> instance.instance_attr
'Hello, Python!'
```

Podemos modificar la instancia .__dict__ dinámicamente. Esto significa que podemos cambiar el valor de los atributos de instancia existentes mediante .__dict__. Incluso podemos agregar nuevos atributos a una instancia usando su diccionario .__dict__.

EJERCICIO 2.3